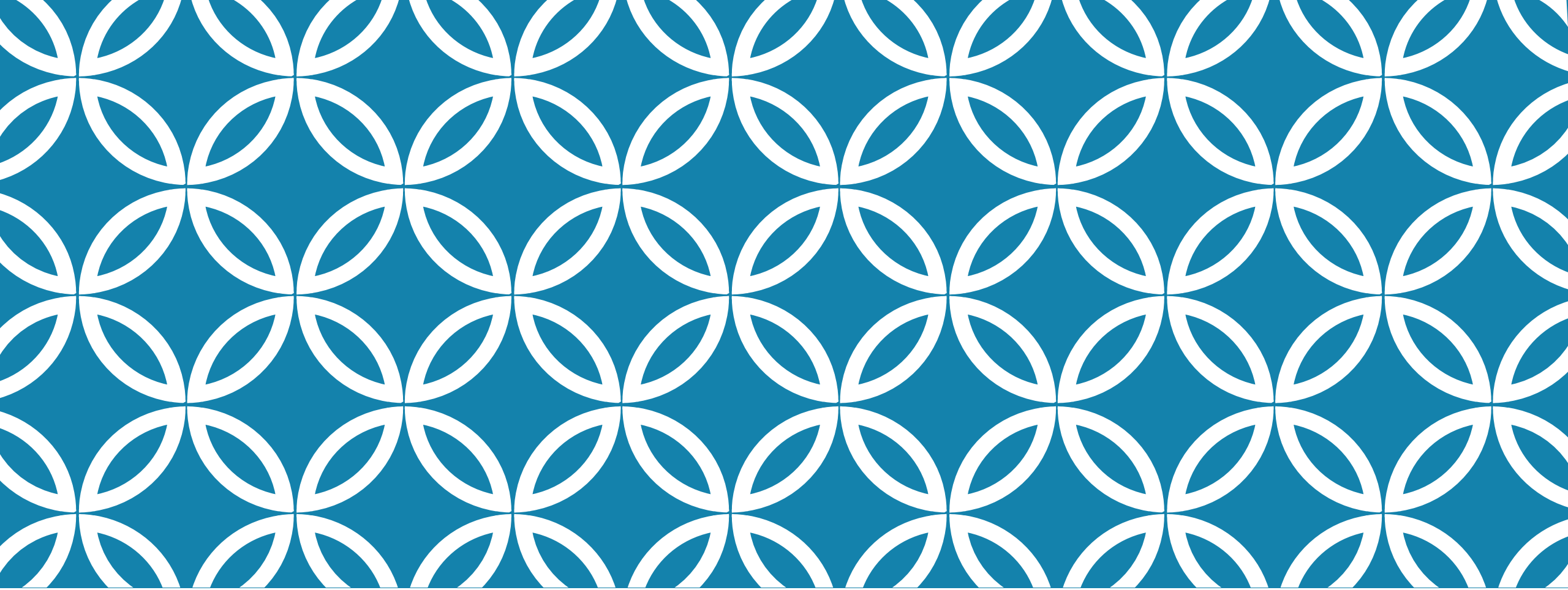


NESNE TABANLI PROGRAMLAMA-I

Öğr. Gör. Erhan AKAGÜNDÜZ

NESNE TABANLI PROGRAMLAMA (OOP) NEDİR?

- OOP, yazılım geliştirme için kullanılan bir yöntemdir ve gerçek dünyadaki nesneleri yazılım ortamına taşır.
- Her nesnenin kendine ait özellikleri ve işlevleri vardır.
- Python, OOP'yi öğrenmeyi kolaylaştıran bir dil olarak öne çıkar.
- OOP bilmeden de Python ile program yazabilirsiniz, ama büyük projelerde zorunlu hale gelir.
- Bu yöntem sayesinde kodlar daha okunabilir, bakımı daha kolay ve sürdürülebilir olur.
- Öğrenmeye karar verirsek, kodlarımızın daha modüler ve tekrar kullanılabilir olmasını sağlarız.

OOP BİLMEK ZORUNDA MIYIM?

- Küçük uygulamalar ve basit scriptler için OOP zorunlu değildir.
- Ancak büyük projelerde, kodları modüllere ayırmadan yönetmek zordur.
- Başkalarının yazdığı kodları anlamak için OOP bilmek büyük kolaylık sağlar.
- Profesyonel bir yazılımcı olmak isteyenler için OOP bilgisi temel bir beceridir.
- Kodun okunabilirliği ve verimliliği OOP ile çok daha yüksek olur.
- Bu yüzden, OOP'yi öğrenmek uzun vadede zaman ve enerji tasarrufu sağlar.

NEDEN PYTHON İLE OOP ÖĞRENİYORUZ?

- Python, OOP öğrenmek için en uygun dillerden biridir.
- Sözdizimi diğer dillere göre daha basittir ve kavramları daha kolay aktarır.
- Java veya C++'ta daha karmaşık kurallar vardır, Python ile işiniz kolaylaşır.
- Python esnek bir dil olduğu için fonksiyonel ve nesne tabanlı kodları birlikte kullanabilirsiniz.
- Python ile OOP öğrenmek, gerçek dünya projelerinde size avantaj sağlar.
- Bu sayede diğer dilleri öğrenirken de OOP mantığını rahatlıkla uygulayabilirsiniz.

OOP'NİN SAĞLADIĞI AVANTAJLAR

- Kodlar daha okunabilir ve anlaşılır olur.
- Projeleri modüllere ayırmak kolaylaşır.
- Kodları değiştirmek ve yeni işlevler eklemek daha hızlı olur.
- Bakım ve hata ayıklama süreçleri daha düzenli hale gelir.
- Modülleri tekrar başka projelerde kullanabilirsiniz.
- İşbirliği gerektiren büyük projelerde ekip çalışmasını kolaylaştırır.

OOP VE BÜYÜK PROJELER

- Büyük projelerde iş bölümü yapmak zorunludur.
- OOP, projeyi parçalara ayırarak yönetmeyi sağlar.
- Kodları modüller halinde yazmak, projeyi daha sürdürülebilir yapar.
- Başka bir yazılımcının kodunu anlamak kolaylaşır.
- Hataları tespit etmek ve düzeltmek hızlı olur.
- Bu nedenle profesyonel yazılım geliştirmede OOP vazgeçilmezdir.

NESNELER – GERÇEK HAYATTAN YAZILIMA

- OOP’de “nesne” gerçek hayattaki varlıkların soyutlanmış hâlidir.
- Örneğin bilgisayar, telefon veya masa bir nesnedir.
- Her nesnenin özellikleri (renk, marka, boyut) ve işlevleri (açma, kapama, yazdırma) vardır.
- Yazılımda bu nesneleri sınıflar aracılığıyla tanımlarız.
- Bu sayede kodlarımız gerçek hayattaki nesneler gibi organize olur.
- Böylece yazılım tasarımı daha doğal ve anlaşılır hâle gelir.

NESNELERİN ÖZELLİKLERİ VE İŞLEVLERİ

- **Özellikler:** Nesnenin sahip olduğu veriler veya durumlar.
- **İşlevler:** Nesnenin yapabildikleri, fonksiyonlar veya metotlar aracılığıyla gerçekleştirilir.
- Örneğin bir Araba nesnesinin rengi, markası, modeli özellikleridir.
- Aynı Araba nesnesinin çalışması, hızlanması işlevleridir.
- OOP'de nesneleri oluşturmak için sınıflar kullanılır.
- Sınıflar, benzer özellik ve işlevlere sahip nesneleri bir araya getirir.

SINIF (CLASS) NEDİR?

- Sınıf, nesnelerin şablonu veya tasarımıdır.
- Python’da sınıflar, veri türleri gibi özel bir türdür.
- Önceden öğrendiğimiz listeler, stringler, tuple’lar da aslında sınıf tipidir.
- Sınıflar ortak özellikleri ve işlevleri bir araya getirir.
- Örneğin tüm arabaların marka, model ve renk özellikleri vardır.
- Bu şablon üzerinden nesneler oluşturulur ve somut hâle getirilir.

PYTHON'DA SINIF OLUŞTURMA

- Sınıf oluşturmak için class anahtar kelimesi kullanılır.
- Sınıf oluşturmak için 2 yöntem vardır:
  - class SınıfAdı:
  - class SınıfAdı( ):
- Parantezli yapı, kalıtım durumlarında gereklidir.
- Parantezsiz yapı, sadece basit sınıf tanımları için kullanılır.
- Sınıf şablonu içinde nitelikler ve metotlar tanımlanır.
- Sınıflar sayesinde kodlarınız organize ve okunabilir olur.

BASİT BİR ARABA SINIFI

- **pass** anahtar kelimesi sınıfın şimdilik boş olduğunu gösterir.
- Bu şekilde bir sınıf tanımlayıp daha sonra özellik ve işlev ekleyebilirsiniz.
- ***Sınıf*** oluşturmak, ***nesne*** üretmenin ilk adımıdır.
- OOP'de sınıflar, gerçek dünyadaki nesnelere karşılık gelir.
- Her nesne sınıfın bir örneği olarak oluşturulur.
- Bu sayede kod tekrarı azaltılır ve düzen sağlanır.

```
class Araba:
    pass
```

SINIF NİTELİKLERİ

- Sınıf nitelikleri, nesnelerin sahip olacağı özelliklerdir.
- Örneğin: marka, renk, plaka.
- Python'da sınıf içinde değişken olarak tanımlanırlar.
- Bu nitelikler tüm nesneler için geçerlidir.
- Daha sonra nesnelere özel değerler atanabilir.

```
class Araba:
    marka = ""
    renk = ""
    plaka = ""
```

NESNE OLUŞTURMA

- Sınıftan nesne oluşturmak için sınıf adı kullanılır.
- Nesne oluşturulduğunda sınıfın tüm niteliklerini miras alır.
- Nesneye ait özellikler daha sonra atanabilir.
- Böylece sınıf soyut bir kavram, nesne somut bir varlık olur.
- Bu ayrım OOP'nin temel prensiplerinden biridir.

```
araba1 = Araba()
```

```
araba1.marka = "FORD"
araba1.renk = "Siyah"
```

SINIF VE NESNE ARASINDAKİ FARK

- Sınıf, bir nesnenin genel tasarımıdır, somut bir örneği yoktur.
- Nesne, sınıfın gerçek hayattaki karşılığıdır ve değerleri atanmıştır.
- **Örnek:** Araba() sınıfı genel araba kavramıdır, **araba1** nesnesi ise **FORD** marka somut bir arabadır.
- Sınıf soyuttur, nesne somuttur.
- Bir sınıftan birden fazla nesne oluşturabilirsiniz.
- Bu sayede aynı tasarım farklı değerlerle tekrar tekrar kullanılabilir.

BİRDEN FAZLA NESNE ÖRNEĞİ

- Çıktılar farklı nesnelere farklı özellikler atanabilir.
- Bu, OOP'nin "bir defa yaz, birçok kez kullan" prensibidir.
- Kod tekrarı en aza iner ve proje yönetimi kolaylaşır.
- Nesneler birbirinden bağımsızdır.
- Her nesne kendi özelliklerini saklar.
- Bu sayede esnek ve modüler bir yapı elde edilir.

```
araba1 = Araba()
araba1.marka = "FORD"
araba1.renk = "Siyah"

araba2 = Araba()
araba2.marka = "BMW"
araba2.renk = "Beyaz"
```

NESNELERE İŞLEVLER EKLEMEK

- Nesneler sadece veri taşımaz, işlev de yapabilir.
- İşlevler sınıfın içinde metot olarak tanımlanır.
- Metotlar nesnenin davranışlarını temsil eder.
- self parametresi, metotların hangi nesne üzerinde çalışacağını belirtir.
- Her metot self ile başlamak zorundadır.

```
def hizArttir(self):
    self.hiz += 10
    return self.hiz
```

SELF ANAHTAR KELİMESİ

- **self**, nesnenin kendisine referans verir.
- Dışarıdan metot veya özelliğe ulaşmak için kullanılır.
- Fonksiyon çağrılırken self otomatik eklenir.
- Bu sayede metotlar nesneye bağlı olarak çalışır.
- **self** olmazsa metot hangi nesneye ait olduğunu bilemez.
- Python’da sınıf içi metotlar için her zaman gereklidir.

METOT ÖRNEĞİ

- Metodu çağırdığınızda hız artar.
- **araba.hizArttir()** fonksiyonu ile hız değerini artırabilirsiniz.
- Fonksiyon her çağrıldığında 10 birim hız ekler.
- Bu örnek OOP'nin dinamik yapısını gösterir.
- Kod tekrarı olmadan nesne davranışı değiştirilebilir.
- Programın işlevselliği nesneye bağlı olarak şekillenir.

```
class Araba:
    hiz = 0
    def hizArttir(self):
        self.hiz += 10
        return self.hiz
```

FONKSİYON ÇAĞIRMA ÖRNEĞİ

- **Çıktı:** 20 (her çağrıda +10)
- Fonksiyon tekrar tekrar kullanılabilir.
- Bu esneklik OOP'nin en önemli avantajlarından biridir.
- Kod daha düzenli ve yönetilebilir olur.
- Bir nesnenin işlevleri kendi içindedir.
- Böylece global değişken kullanmadan işlem yapılabilir.

```
araba.hizArttir()
araba.hizArttir()
print(araba.hiz)
```

NESNE OLUŞTURURKEN ÖZELLİK ATAMA

- Nesneyi oluştururken özelliklerini tanımlayabilirsiniz.
- Bu sayede varsayılan değerler atanmış olur.
- Nesneye sonradan değer vermek zorunda kalmazsınız.
- Ancak tüm nesneler aynı değerleri alır.
- İhtiyaç duyarsanız değişiklik yapılabilir.
- Bu yöntem başlangıç değerleri için idealdir.

```
class Araba:
    marka = "FORD"
    renk = "KIRMIZI"
    plaka ="34 GH 100"
    hiz = 20
```

İNİT() FONKSİYONU

- **__init__( )** yapıcı (constructor) fonksiyondur.
- Nesne oluşturulduğunda otomatik çalışır.
- Özellikleri ve başlangıç değerlerini burada tanımlayabilirsiniz.
- Fonksiyon çağrılmaz, nesne oluşturulduğunda aktif olur.
- Python’da nesneleri özelleştirmenin temel yöntemidir.

```
def __init__(self, marka, renk, plaka, hiz):
    self.marka = marka
```

İNİT() ÖRNEĞİ

- İki nesne farklı özelliklerle oluşturulabilir.
- Nesneler kendi değerlerini taşır.
- Sınıf soyut, nesneler somuttur.
- Başlangıç değerleri ile birlikte nesne hazır hale gelir.
- Kod okunabilir ve modüler olur.
- Proje büyüdüğünde yönetimi kolaylaşır.

```
araba1 = Araba("Fiyat","Mavi","25 P 45",30)
araba2 = Araba("Ford","Beyaz","14 YH 566",100)
```

DEFAULT DEĞERLER

- **__init__( )** fonksiyonunda **default** değerler atayabilirsiniz.
- Nesne oluşturulurken değer verilmezse **default** değer kullanılır.
- Bu, hızlı nesne oluşturmayı sağlar.
- Kodun esnekliği artar.
- Gereksiz tekrar önlenir.
- OOP'de esneklik ve modülerlik sağlar.
- Farklı nesneler farklı değerler alabilir.

```
def __init__(self, marka="Fiyat", renk="Mavi", plaka="25 P 45", hiz=30):
```

KISMİ DEFAULT DEĞERLER

- İstediğiniz özellikleri default, istediğinizi kullanıcıya bırakabilirsiniz.
- **None** değeri, boş veya sonradan doldurulacak anlamına gelir.
- Bu sayede sadece gerekli parametreleri belirlemek yeterlidir.
- Kod hem okunabilir hem esnek olur.
- Kullanıcıya seçenek bırakır.
- Büyük projelerde sık kullanılır.
- Esnek nesne yönetimi sağlar.

```
def __init__(self, marka="Fiyat", renk=None, plaka="25 P 45", hiz=30):
```

ÖZET – NESNE VE SINIF İLİŞKİSİ

- **Sınıf:** Genel tasarım, soyut kavram.
- **Nesne:** Sınıfın somut örneği, özellikler atanmış.
- Bir sınıftan birden fazla nesne üretilebilir.
- Nesneler birbirinden bağımsızdır.
- Özellikler ve metotlar nesneye özel çalışır.
- Bu ayrım, OOP'nin temel yapı taşını oluşturur.

ÖRNEK – ARABA SINIFI İLE NESNELER

- İki nesne farklı özellikler alabilir.
- Kod tekrarına gerek yoktur.
- Nesneler bağımsızdır.
- İşlevler nesneye bağlıdır.
- OOP'nin tekrar kullanılabilirlik avantajını gösterir.
- Proje yönetimini kolaylaştırır.

```
araba1 = Araba()
araba1.marka = "FORD"
araba2 = Araba()
araba2.marka = "BMW"
```

METOTLAR VE ÖZELLİKLER

- Metotlar nesnenin davranışlarını tanımlar.
- Özellikler nesnenin durumunu gösterir.
- Metotlar self parametresi ile nesneye bağlanır.
- Özellikler ve metotlar nesnelerin kişiselleştirilmesini sağlar.
- Bu sayede aynı sınıftan çok farklı nesneler üretilebilir.
- Kod okunabilir ve bakımı kolay olur.

ÖRNEK – HIZ FONKSİYONU

- Fonksiyon her çağrıldığında hız artar.
- Metot nesneye bağlı çalışır.
- Kod tekrarını önler.
- Nesneler kendi işlevlerini yönetir.
- Bu OOP'nin dinamikliğini gösterir.
- Kullanıcıya esneklik sağlar.

```
araba.hizArttir()
araba.hizArttir()
print(araba.hiz)  # Çıktı 20
```

ÖZET – OOP'NİN AVANTAJLARI

- Kodun okunabilirliği artar.
- Modülerlik sağlanır.
- Bakımı ve genişletmesi kolaydır.
- Tekrar kullanılabilirlik artar.
- Büyük projelerde iş bölümü kolaylaşır.
- Profesyonel yazılım geliştirme için gereklidir.

PYTHON OOP'NİN TEMEL PRENSİPLERİ

- Nesne ve sınıf kavramı.
- Metotlar ve özellikler.
- Self anahtar kelimesi.
- init() fonksiyonu ile nesne başlatma.
- Default değerler ile esneklik.
- Nesne tabanlı programlama mantığıyla kod yazma.

SONUÇ VE ÖZET

- Python ile OOP öğrenmek kolaydır.
- Sınıf ve nesne mantığı, kodun modüler ve okunabilir olmasını sağlar.
- Metotlar ve nitelikler, nesnelerin davranış ve durumlarını yönetir.
- ***init( )*** fonksiyonu ile başlangıç değerleri atanabilir.
- Default değerler ve None kullanımı esneklik sağlar.
- OOP bilmek, büyük projelerde ve profesyonel yazılım geliştirmede sizi bir adım öne çıkarır.